



Assignment 2

Software Requirements Specification

Student	Perni Bharath Raghavendra
Roll number	150096724139
Course	SEM5 — Software Engineering & Project Management
Selected project	FinSight — Personal Finance Management Application
Repository	https://github.com/bharath-541/FinSight

Prepared from the project repository and assignment requirements

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the scope, behaviours and quality constraints for FinSight. FinSight is a MERN application that helps a user record expenses, manage assets and debts, and compare spending with the 50/30/20 budgeting rule.

1.2 Problem statement

Users need one place to enter transactions, understand category trends and see whether their monthly plan is on track. FinSight addresses this by combining a React dashboard with a protected REST API and a MongoDB data model.

1.3 Scope

In scope	Out of scope for this SRS
Registration and login; expense CRUD; categories; 50/30/20 analysis; assets and debts; dashboard charts; monthly income and profile settings; responsive UI.	Bank-account aggregation, automatic transaction import, investment execution, tax advice, and financial institution APIs.

1.4 Definitions

Term	Meaning
Needs	Essential spending such as rent, utilities, groceries and healthcare.
Wants	Discretionary spending such as entertainment, dining and hobbies.
Savings	Savings, investments and debt repayment within the budgeting model.
Budget status	on_track, warning or off_track, based on category thresholds.
Safe to spend	Advisory amount available for wants under the selected plan.

2. Stakeholders and user classes

Stakeholder	Goal	System interest
Registered user	Record and understand personal finances.	Accurate entries, useful charts, privacy and fast feedback.
Product owner	Provide a practical budgeting workflow.	Adoption, clear status indicators and maintainable scope.
Developer or maintainer	Operate the MERN application safely.	Stable API contracts, modular code and testable services.
Database operator	Keep financial records available and isolated.	Schema integrity, backups and environment configuration.

3. Functional requirements

ID	Requirement	Priority	Acceptance condition
FR-01	The system shall register a user and authenticate login credentials.	Must	Valid credentials return a JWT; invalid credentials return an error.
FR-02	The user shall create, edit, delete and categorise an expense.	Must	The expense is persisted and appears in the selected period and category.
FR-03	The system shall classify expenses into Needs, Wants or Savings.	Must	Category totals use the configured bucket mapping.
FR-04	The dashboard shall calculate 50/30/20 budget progress.	Must	Each bucket shows amount, percentage and status.
FR-05	The user shall manage assets and debts.	Should	Records can be added, updated and removed; net worth reflects changes.
FR-06	The system shall display spending trends and comparisons.	Should	Charts show current-period totals and monthly or category comparisons.
FR-07	The user shall maintain profile and monthly income settings.	Should	Updated settings are used in budget calculations.
FR-08	The system shall isolate one user's financial records from another user.	Must	Protected endpoints filter data by authenticated user ID.
FR-09	The system shall provide a responsive dashboard and dark mode.	Could	Core views remain usable on desktop and mobile widths.

4. Non-functional requirements

ID	Quality attribute	Requirement
NFR-01	Security	Passwords shall be hashed with bcrypt; protected routes shall require JWT authentication.
NFR-02	Security	The API shall apply input validation, CORS controls and rate limiting.
NFR-03	Performance	Dashboard data shall be returned in a form suitable for interactive charts and summary cards.
NFR-04	Usability	Labels, status colours and category names shall be understandable without developer knowledge.
NFR-05	Maintainability	The backend shall keep controllers, models, routes and services separated.
NFR-06	Portability	The client shall build with Vite and the server shall run with Node.js and MongoDB configuration.
NFR-07	Reliability	API errors shall return consistent responses and must not expose secrets.

5. External interfaces

Interface	Description
User interface	React, React Router, Tailwind CSS, charts and calendar views provide the browser experience.
REST API	Axios calls the Express API for authentication, users, expenses, assets, debts, summaries and net worth.
Database	MongoDB with Mongoose stores user, expense, asset, debt and snapshot data.
Runtime configuration	Environment variables hold the database URI, JWT secret and deployment settings.

6. System context and architecture

The user interacts with the React client. The client calls protected Express routes, which use controllers and services to read or write MongoDB documents. Authentication middleware attaches the user identity used for data isolation.

Component	Responsibilities	Representative routes or data
React client	Dashboard, forms, charts, calendar and profile settings.	Pages, components, Context API and Axios services.
Express API	Routing, validation, authentication and response shaping.	/api/auth, /api/user, /api/expenses, /api/assets, /api/debts, /api/summary, /api/net-worth.
Service layer	Budget calculations, financial insights and business rules.	50/30/20 status, safe-to-spend, trends and net-worth snapshots.
MongoDB	Persistent user-scoped records.	Users, Expenses, Assets, Debts, NetWorthSnapshots.

7. Primary use cases

Use case	Precondition	Main flow	Result
Register or log in	User has valid credentials.	Submit credentials → validate → issue JWT.	Authenticated session.
Record expense	User is authenticated.	Enter amount/category/date → validate → save.	Expense appears in totals.
Review budget	Expenses and income exist.	Open dashboard → calculate buckets → render status.	User sees progress and warnings.
Manage net worth	User is authenticated.	Add assets or debts → recalculate snapshot.	Updated net-worth view.

8. Traceability matrix

Requirement	UI area	API or module	Verification idea
FR-01	Auth pages	/api/auth	Register, login, bad credentials and token expiry.
FR-02	Expense form and timeline	/api/expenses	Create, update, delete and user-isolation tests.
FR-04	Dashboard cards and charts	/api/summary	Known fixture totals and status boundary tests.
FR-05	Assets and debts views	/api/assets, /api/debts	CRUD and net-worth recalculation.
FR-06	Trend charts	/api/summary	Current vs previous month fixture comparison.
FR-08	All protected views	Auth middleware	User A cannot read User B records.
NFR-01	Login and API	bcrypt and JWT middleware	Password hash inspection and protected-route checks.
NFR-02	API boundary	Validation, CORS and rate limiting	Malformed input, origin and repeated-request tests.

9. Assumptions and open questions

- A user enters transactions manually; bank synchronisation is outside scope.
- Budget thresholds are based on the 50/30/20 model and may be configurable later.
- Financial insights are advisory and do not constitute regulated financial advice.
- The SRS should be updated if multi-user households, currencies or recurring transactions are added.

Conclusion

FinSight has a coherent product boundary: secure personal finance records, clear budgeting calculations and user-facing insight. This SRS turns the repository description into testable requirements while preserving the project's current MERN architecture.

References

- GitHub repository: <https://github.com/bharath-541/FinSight>
- Repository documentation: docs/API_DOCUMENTATION.md and docs/SYSTEM_ARCHITECTURE.md.